Kalman-Style Gradient Trust Experiments: Scalar vs. Tensor Mode

1 Goal

This experiment compares several baseline neural-network variants against Kalman-style gradient trust variants. The main question is whether adding a Kalman-inspired optimizer wrapper improves test performance, especially when the data become noisier. The final plots report multi-seed mean test accuracy and mean test error with standard-deviation bars.

2 Kalman Gradient Trust Optimizer

Both the scalar and tensor experiments use the same optimizer wrapper, `kalmanalgo.KalmanGradientTrust`. The optimizer keeps a running estimate of the filtered gradient and a running estimate of uncertainty/noise. Intuitively, gradients that look stable over time receive higher trust, while gradients that look noisy or uncertain receive lower trust.

A simplified version of the update is:

$$\bar{g}_t = \bar{g}_{t-1} + K_t(g_t - \bar{g}_{t-1}),$$
$$K_t = \frac{P_t}{P_t + R_t},$$

where g_t is the current gradient, $\bar{g}_t$ is the filtered gradient estimate, P_t is the Kalman covariance/uncertainty state, R_t is the estimated gradient noise, and K_t is the Kalman gain. Larger K_t means the optimizer trusts the current gradient more; smaller K_t means it relies more on the filtered historical estimate.

3 Tensor Mode

Tensor mode is selected with:

```
--diag-mode tensor
```

In tensor mode, the optimizer tracks separate Kalman covariance, gain, and noise estimates for every individual weight or bias entry. For example, if a layer weight matrix has shape $[800, 784]$, then the Kalman state also has shape $[800, 784]$. This is the more fine-grained version because each parameter element can receive its own trust/gain value.

4 Scalar Mode

Scalar mode is selected with:

```
--diag-mode scalar
```

In scalar mode, the optimizer tracks one scalar covariance/gain/noise estimate per parameter tensor. For the same $[800, 784]$ weight matrix, the entire tensor shares one Kalman gain value. The filtered gradient mean remains tensor-shaped, but the uncertainty and gain are shared across all entries in that parameter tensor.

5 Practical Difference

The practical distinction is:

Mode	Kalman gain resolution	Practical meaning
`tensor`	Per weight or bias entry	Most fine-grained, highest state cost
`scalar`	One gain per parameter tensor	Coarser, lower state cost

In short:

```
tensor = per-weight/per-bias Kalman gain
scalar = one Kalman gain per parameter tensor
```

This is not the same as the older BBB `kalman_layer` method. The older method scales gradients using a layer-level trust heuristic. The newer `kalmanalgo` scalar mode is coarser than tensor mode, but it still operates inside the optimizer wrapper on parameter tensors rather than through the BBB layer-trust heuristic.

6 Results

On clean MNIST, BBB + Dropout remains the strongest baseline, and BBB + Dropout + Kalman is essentially tied with it. The newer external `kalmanalgo` variants do not clearly improve the best clean-data result. On noisy FashionMNIST, the strongest observed result is the h800/L2 Standard `kalmanalgo scalar` run, which slightly exceeds the nearby standard MLP and standard MLP + Kalman variants. However, the gains are small, so the result should be interpreted as promising but not decisive without more seeds and architecture checks.

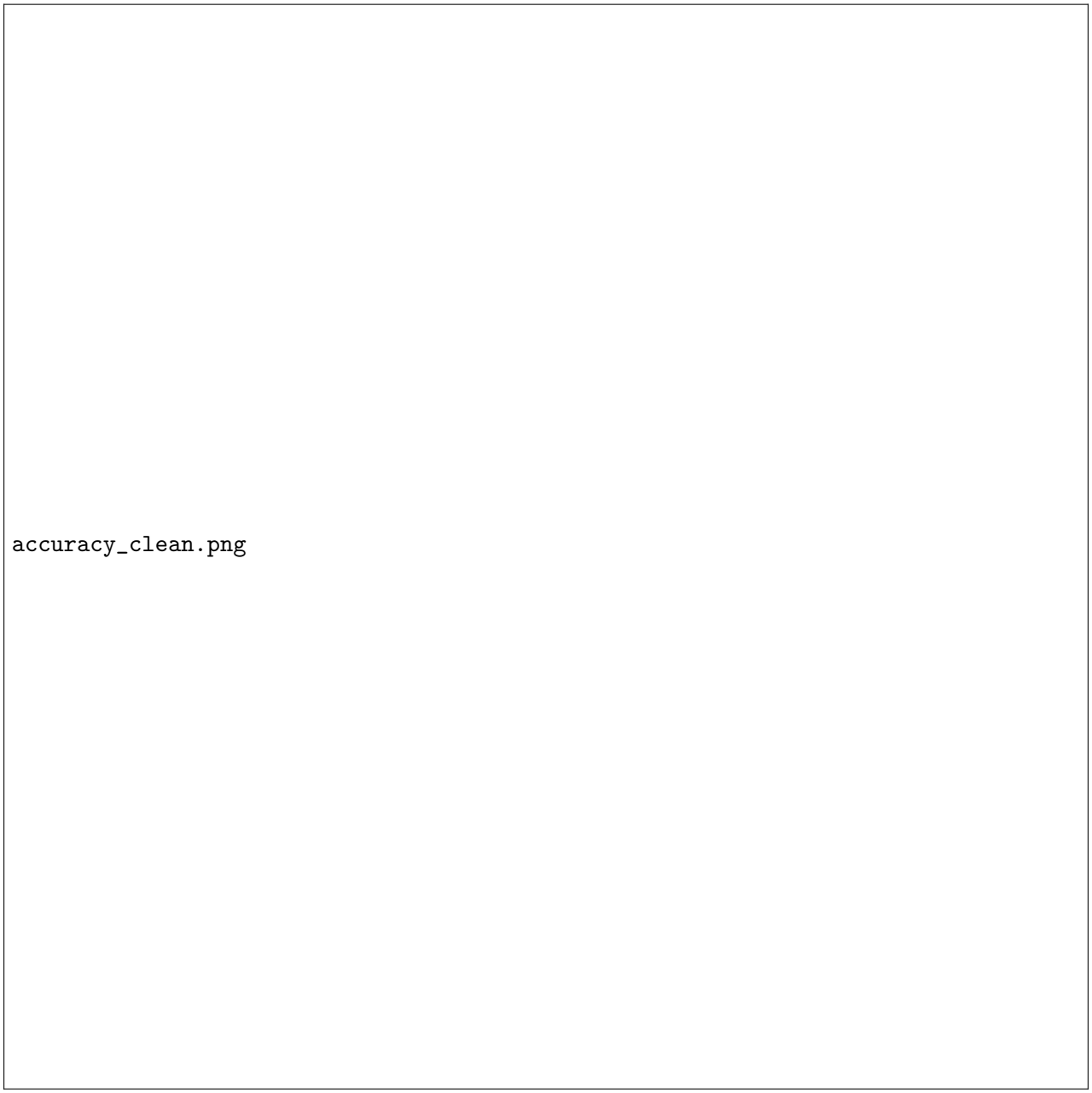

Figure 1: Final multi-seed test accuracy on clean MNIST, including baseline models, older Kalman variants, and external `kalmanalgo` scalar/tensor runs.

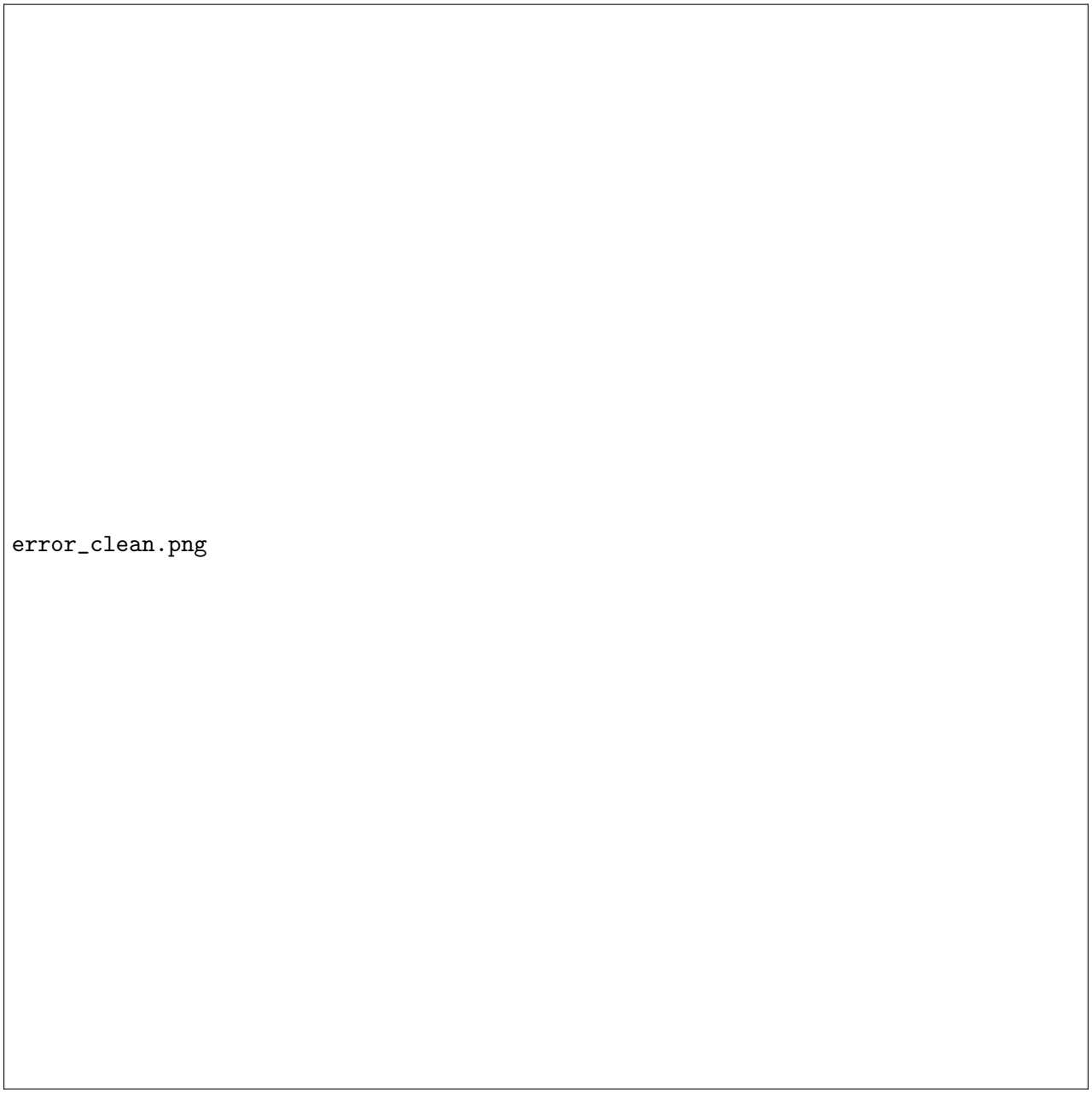

Figure 2: Final multi-seed test error on clean MNIST. Lower error is better.

Figure 3: Final multi-seed test accuracy on noisy FashionMNIST. The h800/L2 Standard `kalmanalgo scalar` run gives the best reported mean accuracy in this plot.

Figure 4: Final multi-seed test error on noisy FashionMNIST. Lower error is better.

7 Takeaway

The scalar and tensor modes test the same Kalman-gradient-trust idea at different resolutions. Tensor mode is more expressive because every parameter entry gets its own uncertainty state, but it also stores much more Kalman state. Scalar mode is cheaper and appears more competitive in the noisy FashionMNIST setting shown here. Overall, the results suggest that Kalman-style trust scaling is not a guaranteed clean-data improvement, but it may be useful when gradients are less stable and the model benefits from a more cautious optimizer.